

Лабораторная работа №8

Целочисленная арифметика многократной точности

Кюнкриков Даниил Саналович

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Представление в основании b	7
3.2	Основные операции	7
4	Выполнение лабораторной работы	10
4.1	Сложение и вычитание	11
4.2	Умножение	13
4.3	Деление с остатком	15
4.4	Результаты	18
5	Выводы	20
	Список литературы	21

Список иллюстраций

4.1	Результат работы программы	19
-----	--------------------------------------	----

Список таблиц

1 Цель работы

Цель работы - Изучить принципы целочисленной арифметики и реализовать базовые операции над целыми числами, записанными в системе счисления с основанием $b \geq 2$, используя поразрядные алгоритмы

2 Задание

1. Реализовать представление неотрицательного целого числа в виде массива цифр в системе счисления с основанием b .
2. Реализовать алгоритмы:
 - сложение,
 - вычитание,
 - умножение столбиком,
 - умножение быстрым столбиком,
 - деление с остатком.
3. Продемонстрировать работу алгоритмов на тестовых примерах.

3 Теоретическое введение

Арифметика многократной точности используется, когда числа не помещаются в стандартные машинные типы. В таких случаях число представляется как массив цифр в основании b , а операции выполняются поразрядно. Классические алгоритмы длинной арифметики подробно изложены в литературе по алгоритмам и численным методам [2; 3].

3.1 Представление в основании b

Неотрицательное целое число задаётся цифрами $u_0, \dots, u_{n-1}$:

$$u = \sum_{i=0}^{n-1} u_i b^i, \quad 0 \leq u_i < b.$$

В реализации удобно хранить число в формате little-endian (младший разряд первым), что упрощает перенос и заём.

3.2 Основные операции

- **Сложение/вычитание** выполняются поразрядно с переносом/заёмом.
- **Умножение «столбиком»** соответствует школьному умножению и имеет сложность $O(nm)$.
- **Деление с остатком** реализуется поразрядно; один из стандартных подходов — деление с нормализацией и оценкой цифр частного (алгоритм

Knuth D) [2].

Практические реализации многоточной арифметики представлены, например, в библиотеке GMP, где используются оптимизации и специализированные алгоритмы для больших размеров [1].

3.2.1 Алгоритм 1 - Сложение

Для каждого разряда складываются числа переносом:

$$s = u_j + v_j + k, \quad w_j = s \bmod b, \quad k = \left\lfloor \frac{s}{b} \right\rfloor.$$

В конце перенос k становится старшим разрядом результата.

3.2.2 Алгоритм 2 - Вычитание

При условии $u \geq v$ вычитание выполняется поразрядно с заемом: - если $u_j - v_j - k < 0$, то прибавляем b и ставим $k = 1$, - иначе $k = 0$

3.2.3 Алгоритм 3 - Умножение столбиком

Классическое умножение - каждый разряд второго множителя последовательно умножается на все разряды первого, суммируя частичные результаты и перенос.

3.2.4 Алгоритм 4 - Быстрый столбик

Вместо вложенных циклов по i и j суммирование ведется по диагоналям: Для каждой суммы индексов s считаем

$$t = \sum u_i \cdot v_{s-i} + \text{carry},$$

после чего выделяем очередную цифру $t \bmod b$ и обновляем перенос $\lfloor t/b \rfloor$.

3.2.5 Алгоритм 5 - Быстрый столбик

Требуется найти частное q и остаток r такие, что:

$$u = qv + r, \quad 0 \leq r < v.$$

В реализации используется поразрядное деление с нормализацией: оценивается очередная цифра частного, затем корректируется, если оценка завышена.

4 Выполнение лабораторной работы

Программная реализация преобразований и нормализации показана на Листинге №1.

4.0.1 Листинг 1 — Представление числа и служебные функции

```
DIGITS = "0123456789ABCDEFGHIJKLMNOPQRSTUVWXYZ"
VAL = {ch: i for i, ch in enumerate(DIGITS)}

def digits_to_str(dig: list[int], b: int) -> str:
    dig = trim(dig[:])
    if len(dig) == 1 and dig[0] == 0:
        return "0"
    out = []
    for d in reversed(dig):
        out.append(DIGITS[d])
    return "".join(out)

def str_to_digits(s: str, b: int) -> list[int]:
    s = s.strip().upper()
    if s == "" or s == "0":
        return [0]
```

```

if s.startswith("-"):
    raise ValueError("Err - <0")
dig: list[int] = []
for ch in reversed(s):
    if ch not in VAL:
        raise ValueError("Err - impossible symbol")
    d = VAL[ch]
    if d < 0 or d >= b:
        raise ValueError("Err - impossible digit")
    dig.append(d)
return trim(dig)

def trim(dig: list[int]) -> list[int]:
    while len(dig) > 1 and dig[-1] == 0:
        dig.pop()
    return dig

```

4.1 Сложение и вычитание

Программная реализация сложения показана на Листинге №2, вычитания - на Листинге №3.

4.1.1 Листинг 2 — Сложение

```

def add_big(u: list[int], v: list[int], b: list[int]) -> list[int]:
    n = max(len(u), len(v))
    w = [0] * (n + 1)
    carry = 0

```

```

for j in range(n):
    uj = u[j] if j < len(u) else 0
    vj = v[j] if j < len(v) else 0
    s = uj + vj + carry
    w[j] = s % b
    carry = s // b

w[n] = carry
return trim(w)

```

4.1.2 Листинг 3 — Вычитание

```

def sub_big(u: list[int], v: list[int], b: int) -> list[int]:
    if cmp_digits(u,v) < 0:
        raise ValueError("Err u should be >= v")
    n = len(u)
    w = [0] * n
    borrow = 0
    for j in range(n):
        uj = u[j]
        vj = v[j] if j < len(v) else 0
        s = uj - vj - borrow
        if s < 0:
            s += b
            borrow = 1
        else:
            borrow = 0
        w[j] = s

```

```
return trim(w)
```

4.2 Умножение

Код функции выполняющей классическое умножение столбиком показан на Листинге №4.

4.2.1 Листинг 4 — умножение столбиком

```
def mul_classic(u: list[int], v: list[int], b: int) -> list[int]:
    u = trim(u[:])
    v = trim(v[:])
    if u == [0] or v == [0]:
        return [0]
    n, m = len(u), len(v)
    w = [0] * (n+m)
    for j in range(m):
        if v[j] == 0:
            continue
        carry = 0
        for i in range(n):
            t = u[i] * v[j] + w[i + j] + carry
            w[i + j] = t % b
            carry = t // b
        w[j + n] += carry
    carry = 0
    for k in range(len(w)):
        t = w[k] + carry
```

```

        w[k] = t % b
        carry = t // b
    while carry > 0:
        w.append(carry % b)
        carry //= b
    return trim(w)

```

Код функции выполняющей быстрое умножение столбиком показан на Листинге №5.

4.2.2 Листинг 5 — быстрый столбик

```

def mul_fast(u: list[int], v: list[int], b: int) -> list[int]:
    u = trim(u[:])
    v = trim(v[:])
    if u == [0] or v == [0]:
        return [0]
    n, m = len(u), len(v)
    w = [0] * (n+m)
    carry = 0
    for s in range(n + m - 1):
        t = carry
        i_min = max(0, s - (m-1))
        i_max = min(n - 1, s)
        for i in range(i_min, i_max + 1):
            t += u[i] * v[s-i]
        w[s] = t % b
        carry = t // b
    w[n + m - 1] = carry

```

```

k = n + m - 1
while w[k] >= b:
    extra = w[k] // b
    w[k] %= b
    k += 1
    if k == len(w):
        w.append(0)
    w[k] += extra

return trim(w)

```

4.3 Деление с остатком

программная реализация деления с остатком показана на Листинге №6.

4.3.1 Листинг 6 — Деление с остатком

```

def div_big(u: list[int], v: list[int], b : int) -> tuple[list[int], list[int]]:
    u = trim(u[:])
    v = trim(v[:])

    if v == [0]:
        raise ZeroDivisionError("Err")
    if u == [0]:
        return [0], [0]
    if cmp_digits(u,v) < 0:
        return [0], u
    if v == [1]:

```

```

    return u, [0]

U = to_be(u)
V = to_be(v)
n = len(V)
m = len(U) - n
uu = [0] + U[:]
vv = V[:]

d = b // (vv[0] + 1)
if d > 1:
    mul_digit_be_inplace(uu, d, b)
    mul_digit_be_inplace(vv, d, b)

n = len(vv)
m = len(uu) - n - 1
q = [0] * (m+1)

v0 = vv[0]
v1 = vv[1] if n > 1 else 0

for j in range (m + 1):
    u0 = uu[j]
    u1 = uu[j + 1]
    numer = u0 * b + u1
    qhat = numer // v0
    rhat = numer % v0
    if qhat >= b:
        qhat = b - 1
        rhat += v0

```



```

if n > 1:
    u2 = uu[j + 2]
    while qhat * v1 > rhat * b + u2:
        qhat -= 1
        rhat += v0
        if rhat >= b:
            break

    borrow = 0
    for i in range(n-1, -1, -1):
        p = qhat * vv[i] + borrow
        idx = j + i + 1
        t = uu[idx] - (p % b)
        borrow = p // b
        if t < 0:
            t += b
            borrow += 1
        uu[idx] = t
    t0 = uu[j] - borrow
    if t0 < 0:
        qhat -= 1
        t0 += b
        carry = 0
        for i in range(n -1,-1,-1):
            idx = j + i + 1
            t = uu[idx] + vv[i] + carry
            if t >= b:
                t -= b

```

```

        carry = 1
    else:
        carry = 0
        uu[idx] = t
        uu[j] = t0 + carry
        if uu[j] >= b:
            uu[j] -= b
    else:
        uu[j] = t0
        q[j] = qhat

r = uu[m + 1 :]
if d > 1:
    r = div_digit_be(r,d,b)

q_le = to_le(q)
r_le = to_le(r)
return q_le, r_le

```

4.4 Результаты

Результат работы программы для заданных параметров показан на Рисунке 4.1.

```
0 - exit
1 - сложение
2 - вычитание
3 - умножение столбиком (классическое)
4 - умножение быстрым столбиком
5 - деление с остатком

Выберите алгоритм (0 для выхода): 1
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u + v = 12

Выберите алгоритм (0 для выхода): 2
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u - v = 0

Выберите алгоритм (0 для выхода): 3
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u * v = 21

Выберите алгоритм (0 для выхода): 4
Введите основание b (2-36)4
Введите u: 3
Введите v: 3
u * v = 21

Выберите алгоритм (0 для выхода): 5
Введите основание b (2-36)4
Введите u: 21
Введите v: 3
q = 3
r = 0

Выберите алгоритм (0 для выхода): █
```

Рисунок 4.1: Результат работы программы

5 Выводы

В ходе работы:

1. Реализованы поразрядные алгоритмы сложения, вычитания, умножения и деления для целых чисел в заданной системе счисления.
2. Представление чисел в виде массива цифр (LE), упростило обработку переносов и заемов.
3. Проверка на тестовых примерах показала корректность работы реализованных операций

Список литературы

1. GNU MP: The GNU Multiple Precision Arithmetic Library (Manual). — Online documentation.
2. *Knuth D. E.* The Art of Computer Programming, Volume 2: Seminumerical Algorithms. — 3-е изд. — Addison-Wesley Professional, 1997. — ISBN 0-201-89684-2.
3. *Shoup V.* A Computational Introduction to Number Theory and Algebra. — 2-е изд. — Cambridge University Press, 2009. — ISBN 0-521-51644-7.